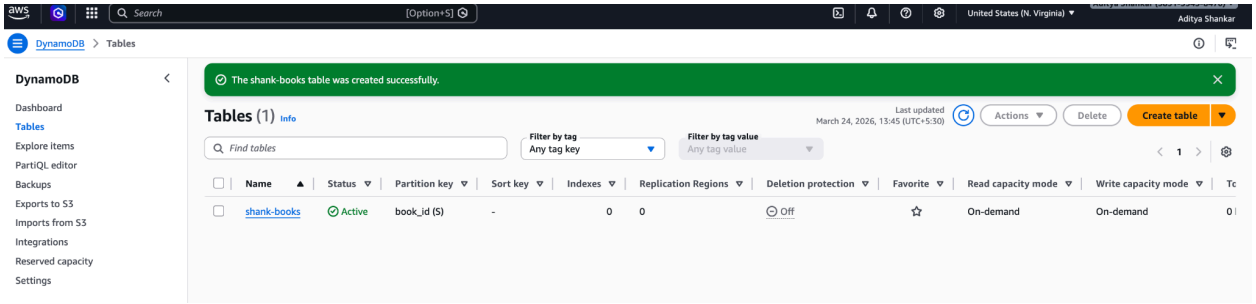
AWS Hands-On Lab: DynamoDB

Date: 24 March 2026 **Account:** Aditya Shankar (5891-5945-8470) **IAM User:** shankdevops
Region: us-east-1 (N. Virginia)

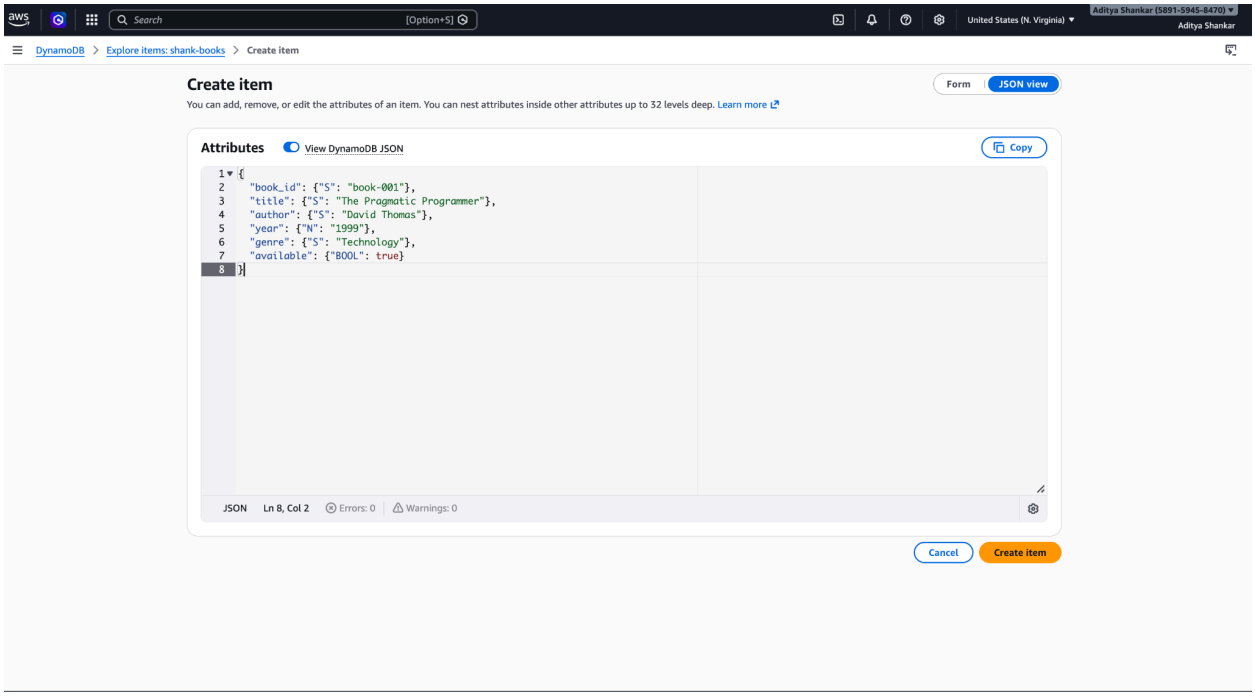
What I did

Step 1 Created a table Created a DynamoDB table named `shank-books` with `book_id` (String) as the partition key. No sort key. Capacity mode set to On-demand pay per request, no capacity planning needed. Table reached Active status immediately.



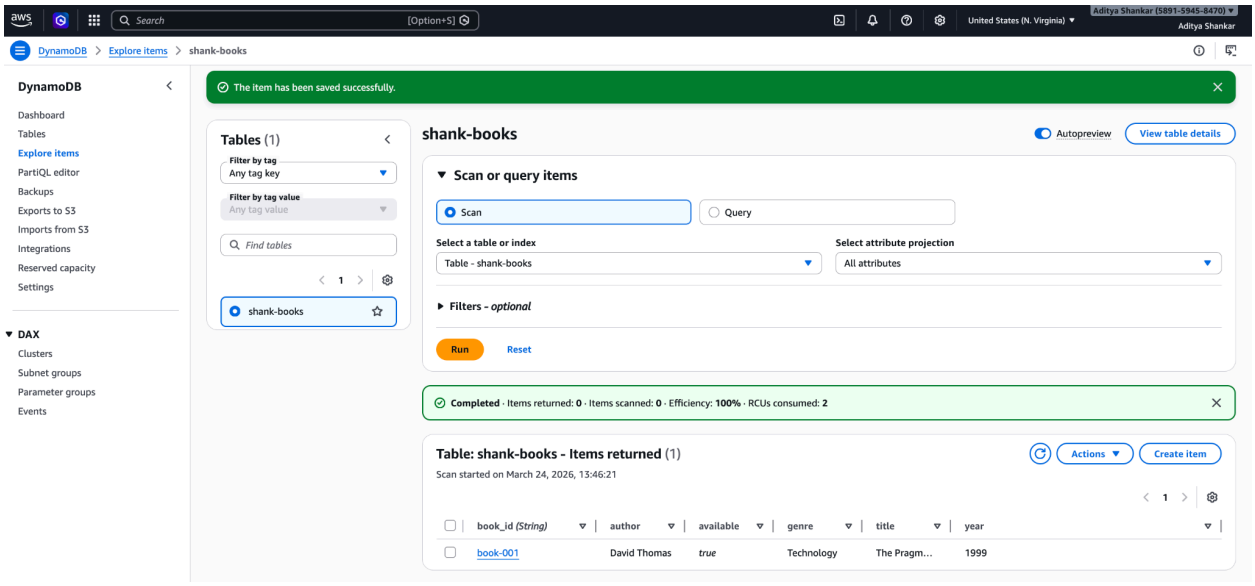
Step 2 Added items via JSON view Added three items using the console's JSON view editor. Each item had a different set of attributes demonstrating DynamoDB's schema-free nature:

- `book-001`: The Pragmatic Programmer had `title`, `author`, `year`, `genre`, `available`
- `book-002`: Clean Code same attributes plus an extra `pages` field the first item didn't have
- `book-003`: Sapiens added `language` and a `ratings` field of type List (`{"L": [...]}`)

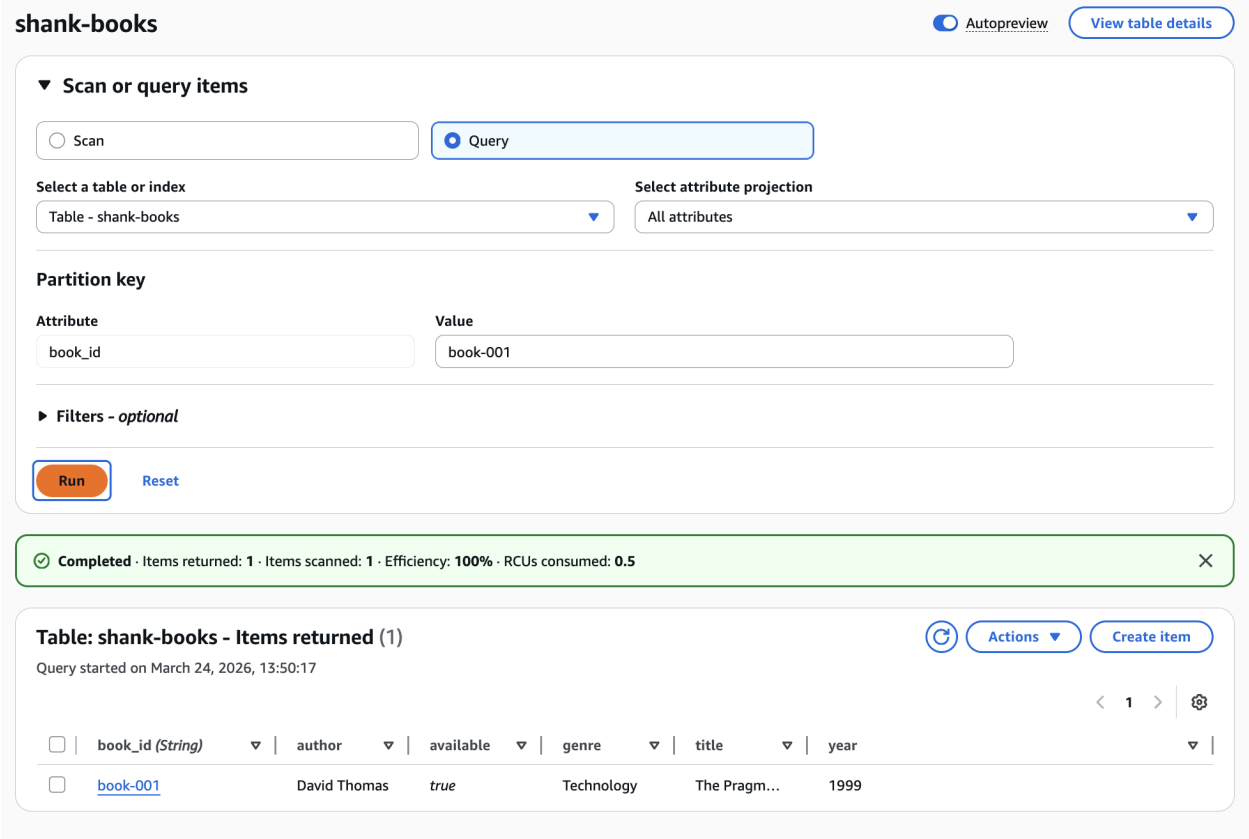


No errors on mismatched attributes between items this is by design. DynamoDB enforces nothing except the partition key.

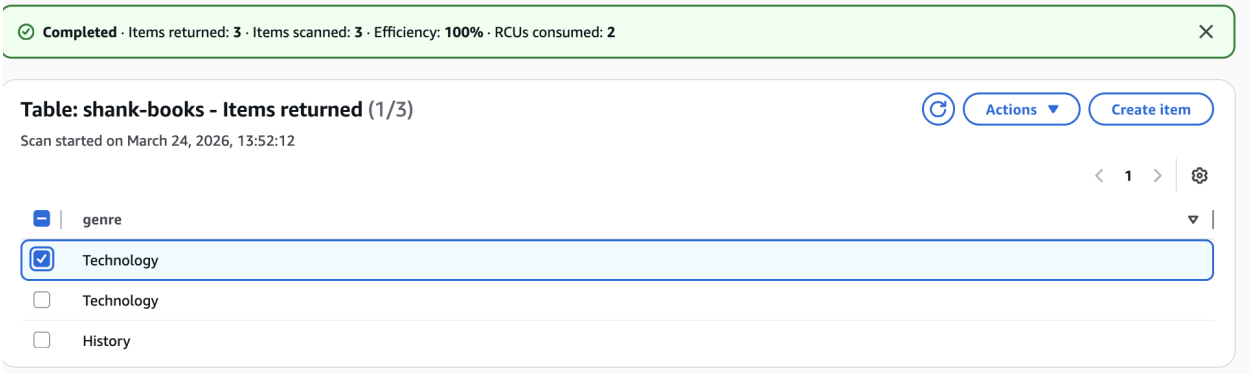
Step 3 Ran a Scan Ran a Scan on the full table. All 3 items returned. Console confirmed: "Items returned: 3 · Items scanned: 3 · RCUs consumed: 2." Scanned all items before returning O(n) behaviour confirmed.



Then filtered the Scan by `genre = Technology`. Two items came back (Clean Code and The Pragmatic Programmer). Importantly, the scan still read all 3 items first the filter reduced the output, not the work done. RCUs consumed remained the same.



Step 4 Ran a Query Switched to Query mode. Queried by `book_id = book-001`. Result: "Items returned: 1 · Items scanned: 1 · RCUs consumed: 0.5." Only one item read, only one returned direct lookup, no full table read. This is the key difference from Scan.



Queried again for `book-003` (Sapiens). Same behaviour 1 scanned, 1 returned, 0.5 RCU.

Key observation from the console stats

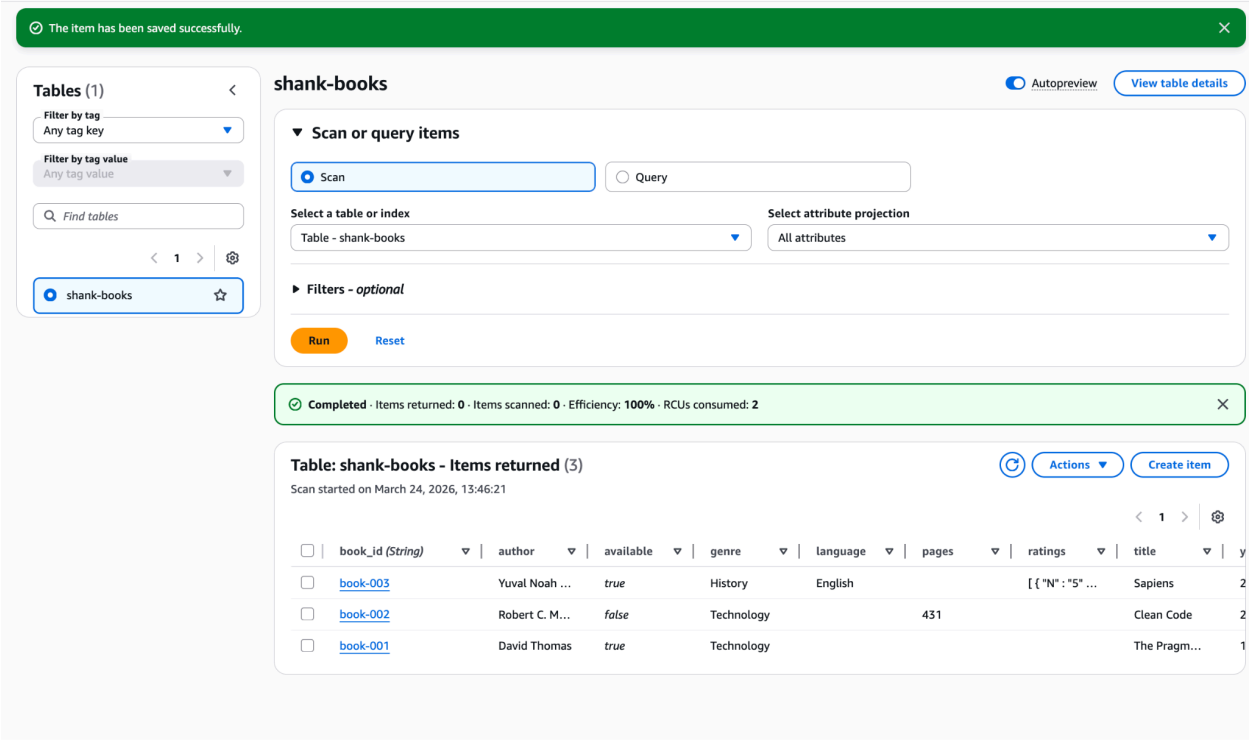
The console shows RCUs consumed on every operation. This made the Query vs Scan difference tangible:

Operation	Items scanned	Items returned	RCUs consumed
Scan (no filter)	3	3	2
Scan (genre filter)	3	2	2
Query (book-001)	1	1	0.5
Query (book-003)	1	1	0.5

At 3 items the difference looks minor. At 3 million items, a Scan would consume millions of RCUs and cost real money. A Query would still consume 0.5 RCU.

Concepts understood

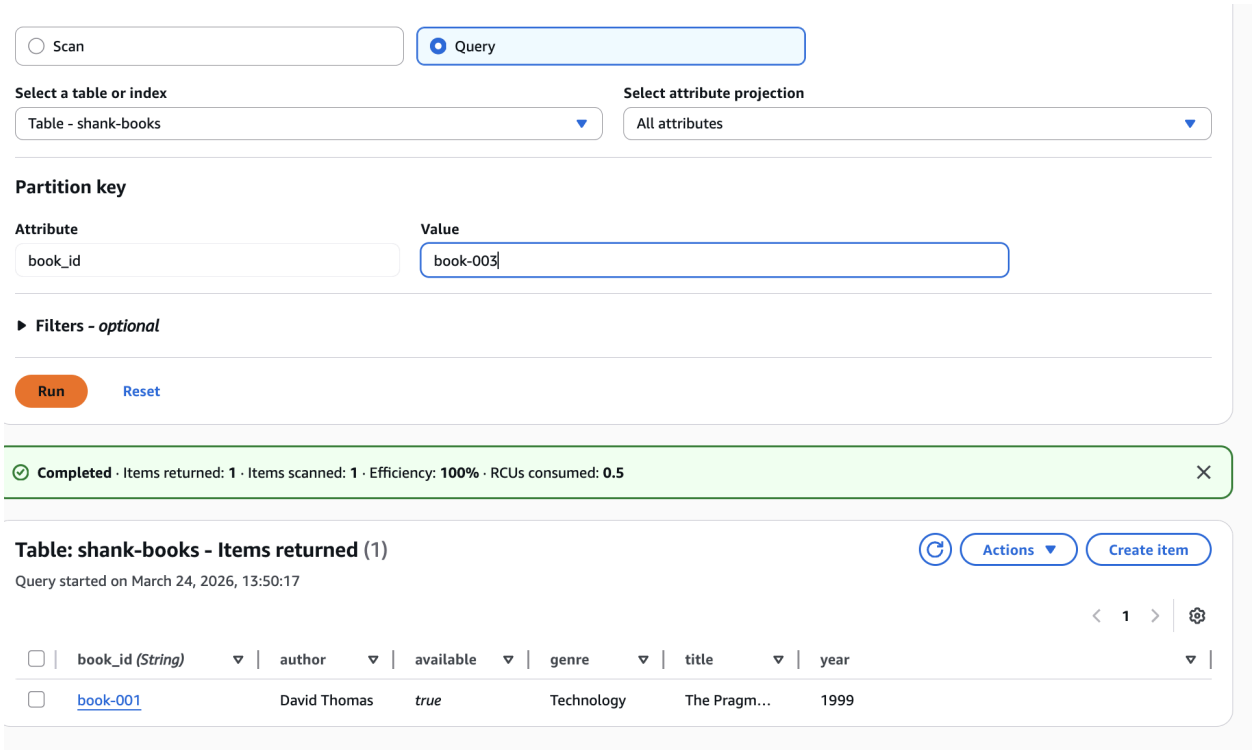
Schema-free storage items in the same table can have completely different attributes. `book-002` had a `pages` field, `book-003` had a `ratings` List neither caused an error, no migration needed.



Type system DynamoDB stores types explicitly in the JSON representation: `{"S": "..."}` for String, `{"N": "..."}` for Number, `{"BOOL": true}` for Boolean, `{"L": [...]}` for List. This is the internal DynamoDB JSON format different from regular JSON.

Query vs Scan the single most important operational concept in DynamoDB:

- Query: knows exactly where to look using the partition key. Cost is proportional to items returned, not table size.
- Scan: reads every item in the table. Cost grows linearly with table size. A filter on a Scan reduces output but not cost.



On-demand capacity no pre-provisioned read/write units. AWS scales automatically. Slightly more expensive per request than Provisioned but requires zero capacity planning correct choice for unpredictable or low traffic workloads.